ASSIGNMENT 05

1. **Create Student class with data members**
   RollNo
   Name
   Course
   and implement following
   • Default constructor
   • Parameterized constructor
   • Destructor
   • Function to accept data
   • Function for display data
   • Write main function and create array of students pointer and store data of multiple students in array.
   • Write Function to find student by Roll No

Code:

```
#include<iostream>
#include<cstring>
using namespace std;

class Student{
   private:
      int rollno;
      char name[20];
      char course[20];
   public:
      Student(){          //default constructor
         rollno = 0;
         name[0] = '\0';
         course[0] = '\0';
      }
      Student(int rno,const char* nm,const char* crs){    //parameterized constructor
         rollno = rno;
         strcpy(name, nm);
         strcpy(course, crs);
      }
      void acceptData(){          //accepting data
            cout<<"Enter roll number: ";
            cin>>rollno;
            cout<<"Enter Name: ";
            cin>>name;
            cout<<"Enter Course: ";
```

```
            cin>>course;
        }
        void displayData(){            //displaying data
            cout<<"Roll number: "<<rollno<<endl;
            cout<<"Enter Name: "<<name<<endl;
            cout<<"Enter Course: "<<course<<endl;
        }
};


int main(){
    int size;
    cout<<"Enter the size of array: "<<endl;
    cin>>size;
    Student s[size];
    //accepting student data in array of objects
    for(int i=0; i<size; i++){
        cout<<"Enter Data For Student "<<i+1<<endl;
        s[i].acceptData();
        cout<<endl;
    }

    //displaying student data from array of objects
     for(int i=0; i<size; i++){
        cout<<"Data of Student "<<i+1<<endl;
        s[i].displayData();
        cout<<endl;
    }
    return 0;
}
```

O/P:-

```
pgcp-ac@dac:~/Batch2/Assignment 05/StudentClass$ g++ studentClass.cpp
pgcp-ac@dac:~/Batch2/Assignment 05/StudentClass$ ./a.out
Enter the size of array:
2
Enter Data For Student 1
Enter roll number: 121
Enter Name: sanket
Enter Course: ac

Enter Data For Student 2
Enter roll number: 122
Enter Name: shubham
Enter Course: dbda

Data of Student 1
Roll number: 121
Enter Name: sanket
Enter Course: ac

Data of Student 2
Roll number: 122
Enter Name: shubham
Enter Course: dbda
```

2. **Create Time class with data members (use operator overloading)**
   hours
   minutes
   seconds
   and implement following
   - **Default constructor**
   - **Parameterized constructor**
   - **Destructor**
   - **Function for display data**
   - **Function to compare two time objects**
   - **Function to find out difference between two time objects**

Code:

```
#include<iostream>
using namespace std;

class Time{
   private:
      int hh, mm, ss;
   public:
      //Parameterized constructor with default values
      Time(int h=0, int m=0, int s=0){
         hh=h;
         mm=m;
         ss=s;
      }
      void acceptTime(){
         cout<<"Enter Time in HH:MM:SS format: "<<endl;
         cin>>hh>>mm>>ss;
      }
      void displayTime(){
         cout<<"HH: "<<hh<<" MM: "<<mm<<" SS: "<<ss<<endl;
      }

      Time operator+(Time t){
         Time tmp;
         tmp.ss = this->ss + t.ss;
         tmp.mm = this->mm + t.mm;
         tmp.hh = this->hh + t.hh;

         //Adjusting seconds
         if(tmp.ss>=60){
            tmp.mm += tmp.ss/60;
            tmp.ss%=60;
```

```
        }

        //Adjusting minutes
        if(tmp.mm>=60){
            tmp.hh += tmp.mm/60;
            tmp.mm%=60;
        }
        return tmp;
    }
    Time operator-(Time t){
        Time tmp;
        tmp.ss = this->ss - t.ss;
        tmp.mm = this->mm - t.mm;
        tmp.hh = this->hh - t.hh;

        //Adjusting seconds
        if(tmp.ss>=60){
            tmp.mm += tmp.ss/60;
            tmp.ss%=60;
        }

        //Adjusting minutes
        if(tmp.mm>=60){
            tmp.hh += tmp.mm/60;
            tmp.mm%=60;
        }
        return tmp;
    }
    bool operator==(Time t){

        return (t.ss==ss && t.mm==mm && t.hh==hh);
    }
    ~Time(){

    }
};
int main(){
    Time t1, t2, t3;
    cout<<"Enter first time: "<<endl;
    t1.acceptTime();
    cout<<"Enter second time: "<<endl;
    t2.acceptTime();
    cout<<"For operator+: "<<endl;
    t3=t1+t2;
```

```
    t1.displayTime();
    t2.displayTime();
    t3.displayTime();
    cout<<"For operator-: "<<endl;
    t3=t1-t2;
    t1.displayTime();
    t2.displayTime();
    t3.displayTime();
    cout<<"For operator==: "<<endl;
    cout<<t1.operator==(t2)<<endl;
    return 0;
}
```

O/P:-

```
pgcp-ac@dac:~/Batch2/Assignment 05$ g++ TimeClass.cpp
pgcp-ac@dac:~/Batch2/Assignment 05$ ./a.out
Enter first time:
Enter Time in HH:MM:SS format:
1
2
3
Enter second time:
Enter Time in HH:MM:SS format:
4
5
6
For operator+:
HH: 1 MM: 2 SS: 3
HH: 4 MM: 5 SS: 6
HH: 5 MM: 7 SS: 9
For operator-:
HH: 1 MM: 2 SS: 3
HH: 4 MM: 5 SS: 6
HH: -3 MM: -3 SS: -3
For operator==:
Not Same
pgcp-ac@dac:~/Batch2/Assignment 05$
```

3. **Create Date class with data members**
   dd
   mm
   yyyy
   and implement following
   • Default constructor
   • Parameterized constructor
   • Destructor
   • Function to find out difference between two dates
   • Function to compare dates
   • Separate header and implementation files
   • Overlaod ==, !=, +, - operator

Code:-

date.h

```
class Date{
   private:
      int dd;
      int mm;
      int yyyy;
   public:
      Date();
      Date(int d, int m, int y);
      void acceptDate();
      void displayDate();
      void operator==(Date d);
      void operator!=(Date d);
      Date operator+(Date d);
      Date operator-(Date d);
      ~Date();
};
```

DateClass.cpp

```
#include "date.h"
#include<iostream>
using namespace std;

Date::Date(){
      dd=0;
      mm=0;
      yyyy=0;

}
```

```
Date::Date(int d, int m, int y){
   dd=d;
   mm=m;
   yyyy=y;
}
void Date::acceptDate(){
   cout<<"Enter DD/MM/YYYY: ";
   cin>>dd>>mm>>yyyy;
}
void Date::displayDate(){
   cout<<dd<<":"<<mm<<":"<<yyyy<<endl;
}
void Date::operator==(Date d){
   if(dd==d.dd && mm==d.mm && yyyy==d.yyyy){
      cout<<"Both Dates are similar."<<endl;
   }
   else{
      cout<<"Both Dates are not similar."<<endl;
   }
}
void Date::operator!=(Date d){
   if(dd!=d.dd && mm!=d.mm && yyyy!=d.yyyy){
      cout<<"Both Dates are Not similar."<<endl;
   }
   else{
      cout<<"Both Dates are similar."<<endl;
   }
}
Date Date::operator+(Date d){
      Date tmp;
      tmp.dd = dd + d.dd;
      tmp.mm = mm + d.mm;
      tmp.yyyy = yyyy + d.yyyy;
      return tmp;
}
Date Date::operator-(Date d){
      Date tmp;
      tmp.dd = dd - d.dd;
      tmp.mm = mm - d.mm;
      tmp.yyyy = yyyy - d.yyyy;
      return tmp;
}
Date::~Date(){
```

```
    //Destructor
}
```

DateClassTest.cpp

```
#include<iostream>
#include "date.h"
using namespace std;

int main(){
    Date d1, d2, d3;
    cout<<"Enter Date for D1: "<<endl;
    d1.acceptDate();
    cout<<"Enter Date for D2: "<<endl;
    d2.acceptDate();
    cout<<"For operator+ :  "<<endl;
    d3 = d1+d2;
    d3.displayDate();
    cout<<endl;
    cout<<"For operator- :  "<<endl;
    d3 = d1-d2;
    d3.displayDate();
    cout<<endl;
    cout<<"For operator== :  "<<endl;
    d1==d2;
    d1!=d2;
    return 0;
}
```

O/P:-

```
pgcp-ac@dac:~/Batch2/Assignment 05/DateClass$ g++ DateClass.cpp DateTest.cpp
pgcp-ac@dac:~/Batch2/Assignment 05/DateClass$ ./a.out
Enter Date for D1:
Enter DD/MM/YYYY: 26
04
2003
Enter Date for D2:
Enter DD/MM/YYYY: 29
03
2003
For operator+ :
55:7:4006

For operator- :
-3:1:0

For operator== :
Both Dates are not similar.
For operator!= :
Both Dates are Not similar.
pgcp-ac@dac:~/Batch2/Assignment 05/DateClass$
```

4. **Create Your own String class (Use class name as MyString) with data members**
   char * p;
   int size;
   Allocate memory dynamically for pointer and get size from user.
   Implement followings
   - **Default constructor**
   - **Parameterized constructor**
   - **Copy Constructor**
   - **Destructor for clean-up (use delete compulsory)**
   - **Display function.**
   - **Separate header and implementation files**

Code:-

myString.h

```
class myString{
   private:
         char* p;
         int size;
   public:
         myString();                        //default constructor
         myString(const char* p,int size);    //parameterised constructor
         myString(myString &ms1);             //copy constructor
         // void acceptString();
         void displayString();
         ~myString();                        //destructor
};
```

myString.cpp

```
#include "myString.h"
#include<iostream>
#include<cstring>
using namespace std;

myString::myString(){
   p = new char;
   *p = '\0';
   size = 0;
}

myString::myString(const char* str, int s){
   p = new char[strlen(str)+1];
   strcpy(p,str);
   size = s;
}
```

```
// void myString::acceptString(){
//     cout<<"Enter string: "<<endl;
//     cin>>p;
// }

myString::myString(myString &ms1){          //copy constructor
    this->p=new char[strlen(ms1.p)+1];
    strcpy(this->p,ms1.p);
    size = ms1.size;
}

void myString::displayString(){
    for(int i=0; i<size; i++){
        cout<<p[i];
    }
    cout<<endl;
}

myString::~myString(){
    if(p){
        delete[] p;
    }
}
```

myStringTest.cpp

```
#include "myString.h"
#include<cstring>
#include<iostream>
using namespace std;

int main(){
    myString ms1("shubham", 7);
    ms1.displayString();
    myString ms2(ms1);
    ms2.displayString();
    return 0;
}
```

O/P:-

```
pgcp-ac@dac:~/Batch2/Assignment 05/MyString Class$ g++ myString.cpp myStringTest.cpp
pgcp-ac@dac:~/Batch2/Assignment 05/MyString Class$ ./a.out
shubham
shubham
pgcp-ac@dac:~/Batch2/Assignment 05/MyString Class$
```

5. **Create Player class with data members**
   name
   age
   country
   Implement followings
   • Default constructor
   • Parameterized constructor
   Function to Sort Array of Player objects (implement as global, sorting should be one name and age)
   • Accept Function for data input
   • Display function for data display
   • Separate header and implementation files
   • In main function create array of player objects and pass it to Sort function and print sorted
   player list(sort on name)

Code:-

Player.h

```
class Player{
   private:
      char* name;
      int age;
      char* country;
   public:
      Player();
      Player(const char* nm, int ag, const char* cntry);
      void sortArray(Player** p, int size);
      void acceptData(Player** p, int size);
      void displayData(Player** p, int size);
};
```

Player.cpp

```
#include "Player.h"
#include<iostream>
#include<cstring>
using namespace std;

Player::Player(){
   //char* name = new char[1];
   name = nullptr;
   age = 0;
   //char* country = new char[1];
   country = nullptr;
```

```
}

Player::Player(const char* nm, int ag, const char* cntry){
    name = new char[strlen(nm)+1];
    strcpy(name, nm);
    age = ag;
    country = new char[strlen(cntry)+1];
    strcpy(country, cntry);
}

void Player::acceptData(Player** p, int size){
    //char* nm = new char[1];
    char nm[100] = {0};
    int a = 0;
    //char* cntry = new char[1];
    char cntry[20] = {0};

    for(int i=0; i<size; i++){
        cout<<"Enter name: "<<endl;
        cin>>nm;
        cout<<"Enter age: "<<endl;
        cin>>a;
        cout<<"Enter country: "<<endl;
        cin>>cntry;

        p[i] = new Player(nm, a, cntry);
    }
}

void Player::displayData(Player** p, int size){

    for(int i=0; i<size; i++){
        cout<<"name: "<<p[i]->name<<endl;
        cout<<"age: "<<p[i]->age<<endl;
        cout<<"country: "<<p[i]->country<<endl;
    }
}

void Player::sortArray(Player** p, int size){
    Player *temp;
    for(int i=0; i<size-1; i++){
        for(int j=0; j<size-i-1; j++){
            if(strcmp(p[j]->name, p[j+1]->name) > 0){
                temp = p[j];
```

```
            p[j] = p[j+1];
            p[j+1] = temp;
        }
        else if(strcmp(p[j]->name, p[j+1]->name)==0){
            if(p[j]->age > p[j+1]->age){
                temp=p[j];
                p[j]=p[j+1];
                p[j+1]=temp;
            }
        }
    }
  }


}
```

PlayerTest.cpp

```
#include "Player.h"
#include<iostream>
#include<cstring>
using namespace std;

int main(){
  int n;
  cout<<"Enter size of player array: "<<endl;
  cin>>n;
  Player** parr = new Player*[n];
  Player p;
  p.acceptData(parr, n);
  cout<<"-------------------------------"<<endl;
  p.displayData(parr,n);
  cout<<"-------------------------------"<<endl;
  p.sortArray(parr, n);
  cout<<"-------------------------------"<<endl;
  p.displayData(parr,n);
  cout<<"-------------------------------"<<endl;
  return 0;
}
```

O/P:-

```
pgcp-ac@dac:~/Batch2/Assignment 05/Player$ g++ Player.cpp PlayerTest.cpp
pgcp-ac@dac:~/Batch2/Assignment 05/Player$ ./a.out
Enter size of player array:
4
Enter name:
sanket
Enter age:
32
Enter country:
india
Enter name:
shubham
Enter age:
23
Enter country:
india
Enter name:
sanket
Enter age:
21
Enter country:
uk
Enter name:
shubhan
Enter age:
22
Enter country:
uk
------------------------------
name: sanket
age: 32
country: india
name: shubham
age: 23
country: india
name: sanket
age: 21
country: uk
name: shubhan
age: 22
country: uk
------------------------------
------------------------------
name: sanket
age: 21
country: uk
name: sanket
age: 32
country: india
name: shubham
age: 23
country: india
name: shubhan
age: 22
country: uk
------------------------------
pgcp-ac@dac:~/Batch2/Assignment 05/Player$
```